machine? A one-armed bandit! Get it? It has one arm (a lever) and it generally steals your money. We could call our situation a 10-armed bandit problem, or an *n*-armed bandit problem more generally, where *n* is the number of slot machines. While this problem sounds pretty fanciful so far, you'll see later that these *n*-armed bandit (or multi-armed bandit) problems do have some very practical applications.

Let's restate our problem more formally. We have *n* possible actions (here *n* = 10) where an action means pulling the arm, or lever, of a particular slot machine, and at each play (*k*) of this game we can choose a single lever to pull. After taking an action (*a*) we will receive a reward, R_k (reward at play *k*). Each lever has a unique probability distribution of payouts (rewards). For example, if we have 10 slot machines and play many games, slot machine #3 may give out an average reward of $9 whereas slot machine #1 only gives out an average reward of $4. Of course, since the reward at each play is probabilistic, it is possible that lever #1 will by chance give us a reward of $9 on a single play. But if we play many games, we expect on average that slot machine #1 will be associated with a lower reward than #3.

Our strategy should be to play a few times, choosing different levers and observing our rewards for each action. Then we want to only choose the lever with the largest observed average reward. Thus, we need a concept of expected reward for taking an action (*a*) based on our previous plays. We'll call this expected reward $Q_k(a)$ mathematically: you give the function an action (given we're at play *k*), and it returns the expected reward for taking that action. This is shown formally here:

Table 2.2. The expected reward calculation in math and pseudocode

Math	Pseudocode

$Q_k(a) = \frac{R_1 + R_2 + ... + R_k}{k_a}$	`def exp_reward(action, history):`
	`  rewards_for_action = history[action]`
	`  return sum(rewards_for_action) /`
	`  len(rewards_for_action)`